

Electron

A hierarchical place-and-route suite

- [What Electron is](#)
- [How it is built](#)
 - [Fragments, not modules](#)
 - [Three tools from one source](#)
 - [Everything runs in the container](#)
- [The data model](#)
 - [Units are the recurring bug class](#)
 - [Three pin stores, easily confused](#)
- [The flows](#)
 - [Flat](#)
 - [Hierarchical](#)
 - [Synthesis](#)
- [Commands worth knowing](#)
- [Adding a command](#)
- [Traps](#)
 - [\\$TOP_MODULE and the global are different variables](#)
 - [Two DEFs of one design are not comparable byte for byte](#)
 - [A control run is the only way to attribute a difference](#)
 - [Verify placement independently](#)
 - [An instance name is not a regular expression](#)
- [Confidentiality](#)

What Electron is

Electron is a place-and-route suite written in Perl, licensed AGPL-3.0, forked from efabless/proton. It takes a gate-level netlist and a technology library and produces a placed, legalised and routed design, written out as DEF.

What distinguishes it from a flat place-and-route flow is that it keeps the design's **module hierarchy** and places into it: blocks are arranged inside their parent, then their contents are arranged inside them, top down. A flat placer sees one undifferentiated sea of cells; Electron sees the structure the designer wrote, and can use it.

This document describes how the suite is put together, the data it holds, the flows it supports, and the traps that have cost real time.

How it is built

Fragments, not modules

The files under `PARSER/`, `PLACER/`, `TSTGEN/`, `GUI/`, `ALGO/`, `DB/` and the rest are **not Perl modules**. They are fragments required into a single namespace by the wrappers in `UTILS/*.nopath`. There is no `use strict`, and almost everything is a package global.

The practical consequences matter more than the style:

- A subroutine defined in any fragment can be called from any other.
- A global set in one fragment is visible everywhere.
- `perl -c` on a fragment proves very little, because the names it uses are declared by whatever required it.

Three tools from one source

```
make      -> electron      flat flow
           -> electron_hier hierarchical flow
           -> electron_proto prototyping flow
```

`make` prepends `$BEEHOME` and a `use lib` line to each `.nopath` wrapper and stamps a release ID taken from the commit count. The three built scripts are gitignored and change on every commit — build them after cloning rather than tracking them.

Everything runs in the container

The suite runs inside `INSTALL/podman/pysparkpp.sif`. The host cannot run it: there is no Tk, so every GUI fragment and all three wrappers fail to compile; no scipy, so the placers die; no synthesis or routing tools, so anything that shells out fails.

```
make check      both checks below
make check-load loads all three tools for real -- the one that
                 matters
make check-syntax perl -c over every required fragment (advisory)
make app        an interactive shell in that image
```

A pre-push hook runs `make check` and refuses the push if a tool fails to load. It is committed at `INSTALL/hooks/pre-push` and has to be enabled once per clone, because `.git/hooks` is not version controlled:

```
git config core.hooksPath INSTALL/hooks
```

Use **bash -c**, not **bash -lc**. A login shell sources the host `~/.bashrc` through the bound home directory, which can put a different Python ahead of `/usr/bin` — and that one has no scipy, so a placer dies with “No module named scipy” inside an image that has it.

The data model

Electron holds the design in several global stores. Knowing which one a command reads is usually the key to understanding what it does.

store	holds	units
%CADB	flat instance locations	DBU
%PLDB	cell sizes, LEF pin rectangles, layer pitch	microns
%NETS_AL READY	flat connectivity	—
FLOORPLAN_ALREADY{id}	the flat floorplan: size, pins, instances	DBU
PSEUDO_FLOORPLAN_ALREADY	the hierarchical floorplan	DBU on disk, microns to callers
PORTS_ALREADY{module}{port}	flat ports	DBU

Units are the recurring bug class

Half the stores are in database units and half in microns. Never hardcode the scale factor. \$GLOBAL->dbfGlobalGetDBU is authoritative; \$DEF_DATABASE_UNIT is a global that set_inst_box overwrites mid-session, and \$DBSCALEFACTOR is captured once when a GUI view is built. The two diverge, and that divergence was the cause of a long-standing flyline offset in the flat view.

Three pin stores, easily confused

- **PORTS_ALREADY{module}{port}** — flat ports. What the flat view and write_def read. A newly created port defaults to location (0,0) and side "w", so an unplaced port reports itself on the left edge rather than reporting that it is unplaced.
- **FLOORPLAN_ALREADY{id} pins** — the flat floorplan's pins, which the guide-

driven pin placement edits.

- **PSEUDO_FLOORPLAN_ALREADY** — the hierarchical pins, which `hier_place` and `read_pseudo_nodefile` write.

Nothing bridges the hierarchical side to the flat side automatically. The chain is:

```
commit_module --physical_only  
-> dbfTstgenUpdateFlplanByID  
-> hier2flat  
-> PORTS_ALREADY, %CADB
```

`hier2flat` is global: it rebuilds from the whole hierarchy, not just the module that was edited.

The flows

Flat

```
read_config_file -config <library.config> -foundary <fab> -technode
<node> -layer <n>
read_verilog -v design.vg
set TOP_MODULE design
elaborate
set_floorplan_parameters -ASPECT_RATIO 1 -UTILIZATION 50
set_floorplan
place_flat_design
legalize_flat
write_def -output design.def --overwrite
```

Hierarchical

The hierarchical flow places blocks into their parent, commits that arrangement, then descends:

```
edit_module --top          build the pseudo model from the netlist
hier_place -kinds INST,PORT -batch 400
commit_module --physical_only
hier2flat --physical
```

`hier_place_all` drives that loop over the whole tree automatically.

The walk must go strictly top down. `dbfTstgenEditModule(M)` deletes the pseudo model of *M* and of every hierarchical child of *M* before reloading. A child's size comes from the parent's committed floorplan, so the order is: parent placed, parent committed, then child. Reopening an ancestor mid-walk loses the children below it.

-batch is not optional. `hier_place` is a graphical placer in its own window; without `-batch` each module opens one and waits for a human.

Synthesis

Two commands, for two different needs.

synthesize produces a flat gate-level netlist. One yosys run, hierarchy gone. Right for a small design or a routability experiment.

hier_synthesis produces a *hierarchical* netlist, which is what `hier_place` needs. The front end keeps one module per instance, which on a large design means hundreds of thousands of modules that are only a few hundred distinct structures. Those are grouped, ranked by depth, and each distinct module is synthesised once, deepest first, reading the levels below it as blackboxes:

```
read_library_config -config <library.config> -foundary <fab> -technode  
<node> -layer <n>  
get_std_cell_libs -vt <flavour> -corner <pattern> --first --set  
hier_synthesis -rtl <filelist> -top <module> -out synth -netlist  
design.vg.gz --read
```

-flatten_below <cells> controls how much hierarchy survives: a module stays a boundary only when its subtree is at least that many cells. This is a real trade. A boundary is a wall the mapper cannot optimise across, and on designs with deep, fine-grained hierarchy it has measured as much as 40% of the cell count. On designs whose blocks are wide and shallow it costs almost nothing.

Commands worth knowing

command	does
<code>read_config_file</code>	read the library config; write the LEF loader
<code>read_library_config</code>	record liberty paths from the config
<code>get_std_cell_libs</code>	pick a liberty by VT flavour and corner
<code>read_verilog,</code> <code>elaborate</code>	read a netlist and build the databases
<code>set_floorplan</code>	die area and rows
<code>edit_module</code>	open a module's pseudo model
<code>hier_place,</code> <code>hier_place_all</code>	arrange blocks and ports
<code>commit_module</code>	write an arrangement back to the floorplan
<code>hier2flat</code>	expand the hierarchy into the flat database
<code>place_flat_design</code>	flat placement
<code>legalize_flat</code>	Abacus legalisation; zero overlaps
<code>write_def,</code> <code>read_def</code>	DEF out and in
<code>synthesize,</code> <code>hier_synthesis</code>	RTL to netlist, flat or hierarchical

Every command takes -h.

Adding a command

1. Register it in **commandsFile**, in *both* hashes: %cmds (command to sub) and %checkArguments. Both live inside sub `initiallize_commands`.
2. require the file in **all three** wrappers under UTILS/.
3. End the file with **1;**. Without it the load dies with “did not return a true value”.
4. **Run make check-load**. `perl -c` does not execute a runtime require, so it reports “syntax OK” on a tool whose fragments cannot load. Running `<cmd> -h` through the built tool is enough to prove the file loads.
5. Message numbers must be unique per prefix: `grep -o "PREFIX : [0-9]*" file | sort | uniq -d`
6. Message prefixes abbreviate the command they belong to. Rename them with the command.

Traps

\$TOP_MODULE and the global are different variables

\$TOP_MODULE and \$GLOBAL -> dbfGlobalGetTOP are not the same and they diverge. edit_module moves \$TOP_MODULE to whatever it opened and leaves the global alone. write_def means \$TOP_MODULE by “the design” — the DESIGN line, the die area, the rows, the pins — while hier2flat reads the global.

Descend the hierarchy, forget to put \$TOP_MODULE back, and write_def emits the last module visited: a block-sized die, no pins, and every component of the real design outside the boundary. It reads as a placement bug and is a name.

Two DEFs of one design are not comparable byte for byte

write_def walks keys %CADB, and Perl randomises hash order per process, so the records come out in a different order every run. **Sort and hash the records instead.** Anything compared through a router additionally needs PERL_HASH_SEED=0 and PERL_PERTURB_KEYS=0, because routers process nets in file order: five runs of one byte-identical legalised DEF have given five different unrouted-net counts.

A control run is the only way to attribute a difference

Before concluding that a change caused a difference, run the unchanged code the same way. Time has been lost to a “regression” that was a tool’s own nondeterminism.

Verify placement independently

Do not trust a command’s own summary. Write the DEF and check it with something else — a rectangle sweep for overlaps, a net-to-pin map for connectivity. Check overlaps in integer database units, never in microns: a float comparison invents a phantom overlap at every row boundary.

An instance name is not a regular expression

Instance names reach code that builds patterns from them. A Verilog **escaped identifier** contains regex metacharacters by nature, and generate blocks produce them in quantity. Quote with \Q . . \E when a name is being used to strip a literal prefix. The failure is sometimes loud and sometimes silent — the silent form leaves cells unplaced with no error at all.

Confidentiality

The repository is public and AGPL-3.0, and work is done against customer designs on foundry PDKs under NDA. **No design name and no library name goes in** — not in a commit message, not in a comment, not in a docstring, not in an example command line.

Allowed: designs under TESTS/, which are the repository's own testcases; nangate; and the library config's own foundry keys, which are functional data rather than prose.

INSTALL/hooks/name-check enforces this, and three hooks call it: pre-commit on the staged content, commit-msg on the message, and pre-push on every blob and message being published. The push check is the important one — a name added in one commit and removed in a later one is still in the history and still fetched by anyone who clones.

Write what the thing **is**, not what it is called. “A 119,351-module design” is more informative than the name was.